

P1035R1

Input range adaptors

Published Proposal, 2018-10-08

This version:

<https://wg21.link/p1035>

Author:

[Christopher Di Bella](#)

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

P1035 proposes to introduce seven additional range adaptors in the C++20 timeframe.

Table of Contents

1	Acknowledgements
2	Motivation
3	Proposals
3.1	<code>take_while_view</code>
3.1.1	Motivation
3.1.2	Notes
3.1.3	Interface and specification
3.1.3.1	<code>take_while_view</code> constructors
3.1.3.2	<code>take_while_view</code> conversion
3.1.3.3	<code>take_while_view</code> range begin
3.1.3.4	<code>take_while_view</code> range end
3.1.4	<code>take_while_view::__sentinel</code>
3.1.5	<code>take_while_view::__sentinel</code> constructor
3.1.6	<code>take_while_view::__sentinel</code> conversion
3.1.7	<code>take_while_view::__sentinel</code> comparisons
3.2	<code>view::take_while</code>
3.3	<code>drop_view</code>
3.3.1	Motivation
3.3.2	Interface
3.3.2.1	<code>drop_view</code> constructor
3.3.2.2	<code>drop_view</code> conversion
3.3.2.3	<code>drop_view</code> range begin
3.3.2.4	<code>drop_view</code> range end
3.3.2.5	<code>drop_view</code> size
3.4	<code>view::drop</code>
3.5	<code>drop_while_view</code>

3.5.1	Motivation
3.5.2	Interface
3.5.2.1	<code>drop_while_view</code> constructors
3.5.2.2	<code>drop_while_view</code> conversion
3.5.2.3	<code>drop_while_view</code> begin
3.5.2.4	<code>drop_while_view</code> end
3.6	<code>view::drop_while</code>
3.7	<code>basic_istream_view</code>
3.7.1	Motivation
3.7.2	Interface
3.7.2.1	Concept <code>StreamExtractable</code>
3.7.2.2	Concept <code>StreamInsertable</code>
3.7.2.3	<code>basic_istream_view</code> constructor
3.7.2.4	<code>basic_istream_view</code> begin
3.7.2.5	<code>basic_istream_view</code> end
3.7.2.6	<code>basic_istream_view::__iterator</code>
3.7.2.7	<code>basic_istream_view::__iterator</code> constructor
3.7.2.8	<code>basic_istream_view::__iterator</code> next
3.7.2.9	<code>basic_istream_view::__iterator</code> value
3.7.2.10	<code>basic_istream_view::__iterator</code> comparison functions
3.8	<code>basic_istream_view</code> range adaptors
3.9	<code>zip_with_view</code>
3.9.1	Motivation
3.9.2	<code>__common_tuple</code>
3.9.3	<code>__iter_zip_with</code>
3.9.4	<code>zip_with_view</code>
3.10	<code>view::zip_with</code>
3.11	<code>view::zip</code>
3.12	Reviewing <code>iter_value_t</code> , etc., and introducing <code>range_value_t</code> , etc.

References

Informative References

§ 1. Acknowledgements

I would like to acknowledge the following people for their assistance with this proposal:

1. Casey Carter, for reviewing all submissions to [\[cmcstl2\]](#) and providing feedback that enabled the proposed range adaptors to have high-quality implementations.
2. Eric Niebler, for providing [\[range-v3\]](#) as a reference implementation.

§ 2. Motivation

[\[P0789\]](#) introduces the notion of a range adaptor and twelve pioneering range adaptors that improve declarative, range-based programming. For example, it is possible to perform an inline, in-place, lazy reverse like so:

```
namespace ranges = std::ranges;
namespace view = std::ranges::view;

// Old
auto i = ranges::find(ranges::rbegin(employees), ranges::rend(employees), "Lovelace", &employee::surname);

// New
auto j = ranges::find(employees | view::reverse, "Lovelace", &employee::surname);
[[assert: i == j]];
```

P1035 recognises that P0789 introduces only a few of the widely experimented-with range adaptors in [\[range-v3\]](#), and would like to add a few more to complete the C++20 phase of range adaptors. To this end, P1035 discusses range adaptors that are related to those in P0789.

§ 3. Proposals

Unless otherwise requested, each sub-subsection below should be polled individually from other sub-subsections. Two major questions are to be asked per range adaptor. It is up to LEWG to decide the exact phrasing, but the author’s intentions are listed below.

- 1. Do we want this range adaptor in C++20?
- 2. As-is?
- 3. With modifications, as suggested by LEWG?
- 4. If we do not want this range adaptor in C++20, do we want it in C++23?
- 5. As-is?
- 6. With modifacitons, as suggested by LEWG?

§ 3.1. `take_while_view`

§ 3.1.1. Motivation

P0789 introduces `take_view`, which rangifies the iterator pair `{ranges::begin(r), ranges::next(ranges::begin(r), n, ranges::end(r))}`. As an example:

```
auto v = std::vector{0, 1, 2, 3, 4, 5};
std::cout << distance(v | view::take(3)) << '\n'; // prints 3
copy(v | view::take(3), ostream_iterator<int>(std::cout, " ")); // prints 0 1 2
copy(v | view::take(distance(v)), ostream_iterator<int>(std::cout, " ")); // prints 0 1 2 3
```

`take_while_view` will provide slightly different functionality, akin to having a sentinel that checks if a certain predicate is satisfied.

Current	Proposed
---------	----------

```

namespace ranges = std::experimental::ranges;

template <ranges::Integral I>
constexpr bool is_odd(I const i) noexcept
{
    return i % 2;
}

struct is_odd_sentinel {
    template <ranges::Iterator I>
    constexpr friend bool operator==(I const& a,
        is_odd_sentinel) noexcept
    {
        return is_odd(*a);
    }

    template <ranges::Iterator I>
    constexpr friend bool operator==(is_odd_sentinel const a,
        I const& b) noexcept
    {
        return b == a;
    }

    template <ranges::Iterator I>
    constexpr friend bool operator!=(I const& a,
        is_odd_sentinel const b) noexcept
    {
        return not (a == b);
    }

    template <ranges::Iterator I>
    constexpr friend bool operator!=(is_odd_sentinel const a,
        I const& b) noexcept
    {
        return not (b == a);
    }
};

int main()
{
    auto v = std::vector{0, 6, 8, 9, 10, 16};
    ranges::copy(
        ranges::begin(v),
        is_odd_sentinel{},
        ranges::ostream_iterator<int>(std::cout, "\n"));
}

```

[Wandbox demo](#)

```

namespace ranges = std::experimental::ra

template <ranges::Integral I>
constexpr bool is_odd(I const i) noexcept
{
    return i % 2;
}

template <ranges::Integral I>
constexpr bool is_even(I const i) noexcept
{
    return not is_odd(i);
}

int main()
{
    namespace view = ranges::view;
    auto v = std::vector{0, 6, 8, 9, 10,
    ranges::copy(
        v | view::take_while(is_even<int>),
        ranges::ostream_iterator<int>(std:
}

```

[Wandbox demo](#)

The former requires that a user define their own sentinel type: something that while not expert-friendly, is yet to be established as a widespread idiom in C++, and providing a range adaptor for this purpose will help programmers determine when a sentinel is `_not_` necessary.

§ 3.1.2. Notes

- There is a slight programmer overhead in the naming: the author felt that both `is_odd_sentinel` and `is_even_sentinel` were applicable names: ultimately, the name `is_odd_sentinel` was chosen because it describes the delimiting condition. An equally valid reason could probably be made for `is_even_sentinel`.
- A sentinel that takes a lambda may be of interest to LEWG. If there is interest in this, a proposal could be made in the C++23 timeframe.

§ 3.1.3. Interface and specification

```
template <View R, class Pred>
requires
    InputRange<R> &&
    std::is_object_v<Pred> &&
    IndirectUnaryPredicate<const Pred, iterator_t<R>
class take_while_view : public view_interface<take_while_view<R, Pred>> {
    template<bool> class __sentinel; // exposition-only
public:
    take_while_view() = default;
    constexpr take_while_view(R base, Pred pred);

    template <ViewableRange O>
    requires constructible-from-range<R, O>
    constexpr take_while_view(O&& o, Pred pred);

    constexpr R base() const;
    constexpr const Pred& pred() const;

    constexpr auto begin() requires !SimpleView<R>;
    constexpr auto begin() const requires Range<const R>;

    constexpr auto end() requires !SimpleView<R>;
    constexpr auto end() const requires Range<const R>;
private:
    R base_; // exposition-only
    semiregular<Pred> pred_; // exposition-only
};

template<class R, class Pred>
take_while_view(R&&, Pred) -> take_while_view<all_view<R>, Pred>;
```

§ 3.1.3.1. `take_while_view` constructors

```
constexpr take_while_view(R base, Pred pred);
```

1. *Effects*: Initialises `base_` with `base` and initialises `pred_` with `pred`.

```
template <ViewableRange O>
requires constructible-from-range<R, O>
constexpr take_while_view(O&& o, Pred pred);
```

2. *Effects*: Initialises `base_` with `view::all(std::forward<O>(o))` and initialises `pred_` with `pred`.

§ 3.1.3.2. `take_while_view` conversion

```
constexpr R base() const;
```

1. *Returns*: `base_`.

```
constexpr const Pred& pred() const;
```

2. *Returns*: `pred_`.

§ 3.1.3.3. `take_while_view` range begin

```
constexpr auto begin() requires !SimpleView<R>;  
constexpr auto begin() const requires Range<const R>
```

1. *Effects*: Equivalent to `return ranges::begin(base_);`.

§ 3.1.3.4. `take_while_view` range end

```
constexpr auto end() requires !SimpleView<R>;  
constexpr auto end() const requires Range<const R>;
```

1. *Effects*: Equivalent to `return __sentinel<is_const_v<decltype(*this)>>(&pred());`.

§ 3.1.4. `take_while_view::__sentinel`

```
template<class R, class Pred>  
template<bool Const>  
class take_while_view<R, Pred>::__sentinel {  
    using Parent = conditional_t<Const, const take_while_view, take_wile_view>;  
    using Base = conditional_t<Const, const R, R>;  
    sentinel_t<Base> end_{}; // exposition-only  
    const Pred* pred_{}; // pred  
public:  
    __sentinel() = default;  
    constexpr explicit __sentinel(sentinel_t<Base> end, const Pred* pred);  
    constexpr __sentinel(__sentinel<!Const> s)  
        requires Const && ConvertibleTo<sentinel_t<R>, sentinel_t<Base>>  
  
    constexpr sentinel_t<Base> base() const { return end_; }  
  
    friend constexpr bool operator==(const __sentinel& x, const iterator_t<Base>& y);  
    friend constexpr bool operator==(const iterator_t<Base>& x, const __sentinel& y);  
    friend constexpr bool operator!=(const __sentinel& x, const iterator_t<Base>& y);  
    friend constexpr bool operator!=(const iterator_t<Base>& x, const __sentinel& y);  
};
```

§ 3.1.5. `take_while_view::__sentinel` constructor

```
constexpr explicit __sentinel(sentinel_t<Base> end, const Pred* pred);
```

1. *Effects*: Initialises `end_` with `end`, and `pred_` with `pred`.

```
constexpr __sentinel(__sentinel<!Const> s)  
    requires Const && ConvertibleTo<sentinel_t<R>, sentinel_t<Base>>;
```

2. *Effects*: Initialises `end_` with `s.end_` and `pred_` with `s.pred_`.

§ 3.1.6. `take_while_view::__sentinel` conversion

```
constexpr sentinel_t<Base> base() const;
```

3. *Effects*: Equivalent to `return end_;`

§ 3.1.7. `take_while_view::__sentinel` comparisons

```
friend constexpr bool operator==(const __sentinel& x, const iterator_t<Base>& y)
```

4. *Effects*: Equivalent to `return x.end_ != y && !(*x.pred_)(*y);`.

```
friend constexpr bool operator==(const iterator_t<Base>& x, const __sentinel& y);
```

5. *Effects*: Equivalent to `return y == x;`.

```
friend constexpr bool operator!=(const __sentinel& x, const iterator_t<Base>& y);
```

6. *Effects*: Equivalent to `!(x == y);`.

```
friend constexpr bool operator!=(const iterator_t<Base>& x, const __sentinel& y);
```

7. *Effects*: Equivalent to `!(y == x);`.

§ 3.2. `view::take_while`

The name `view::take_while` denotes a range adaptor object. Let `E` and `F` be expressions such that type `T` is `decltype((E))`. Then, the expression `view::take_while(E, F)` is expression-equivalent to:

1. `take_while_view{E, F}` if `T` models `InputRange` and if `F` is an object, and models `IndirectUnaryPredicate`.
2. Otherwise `std::ranges::view::take_while(E, F)` is ill-formed.

§ 3.3. `drop_view`

§ 3.3.1. Motivation

`drop_view` is the complement to `take_view`: instead of providing the user with the first `_n_` elements, it provides the user with all `_but_` the first `_n_` elements.

Current (C++17)

```
namespace ranges = std::ranges;
auto i = ranges::next(ranges::begin(employees), 5, ranges::end(employees));
auto j = ranges::find(i, ranges::end(employees), "Lovelace", &employees::surname);
```

```
namespace
auto j = r
```

§ 3.3.2. Interface

```
template<View R>
class drop_view : public view_interface<drop_view<R>> {
    using D = iter_distance_t<iterator_t<R>>; // exposition-only
public:
    drop_view();
    constexpr drop_view(R base, D count);

    template<ViewableRange O>
    requires constructible-from-range<R, O>
    constexpr drop_view(O&& o, D count);

    constexpr R base() const;

    constexpr auto begin() requires !(SimpleView<R> && RandomAccessRange<R>);
    constexpr auto begin() const requires Range<const R> && RandomAccessRange<const R>;
    constexpr auto end() requires !(SimpleView<R>);
    constexpr auto end() const requires Range<const R>;

    constexpr auto size() requires !SimpleView<R> && SizedRange<R>;
    constexpr auto size() const requires SizedRange<const R>;
private:
    R base_; // exposition-only
    D count_; // exposition-only
};

template<Range R>
drop_view(R&&, iter_difference_t<iterator_t<R>>) -> drop_view<all_view<R>>;
```

§ 3.3.2.1. `drop_view` constructor

```
constexpr drop_view(R base, D count);
```

1. *Effects*: Initialises `base_` with `base` and `count_` with `count`.

```
template<ViewableRange O>
requires constructible-from-range<R, O>
constexpr drop_view(O&& o, D count);
```

2. *Effects*: Initialises `base_` with `view::all(std::forward<O>(o))` and `count_` with `count`.

§ 3.3.2.2. `drop_view` conversion

```
constexpr R base() const;
```

3. *Effects*: Equivalent to `return base_`.

§ 3.3.2.3. `drop_view` range `begin`

```
constexpr auto begin() requires !(SimpleView<R> && RandomAccessRange<R>);
constexpr auto begin() const requires Range<const R> && RandomAccessRange<const R>;
```

4. *Effects*: Equivalent to `return ranges::next(ranges::begin(base_), count_, ranges::end(base_))`.

5. `_Remarks_`: In order to provide the amortized constant time complexity required by the Range concept, this function caches the result within the `drop_view` for use on subsequent calls.

§ 3.3.2.4. `drop_view` range end

```
constexpr auto end() requires !(SimpleView<R>);  
constexpr auto end() const requires Range<const R>;
```

6. *Effects*: Equivalent to `return ranges::end(base_);`.

§ 3.3.2.5. `drop_view` size

```
constexpr auto size() requires !SimpleView<R> && SizedRange<R>;  
constexpr auto size() const requires SizedRange<const R>;
```

7. Equivalent to:

```
auto const size = ranges::size(base_);  
auto const count = static_cast<decltype(size)>(count_);  
return size < count ? 0 : size - count;
```

§ 3.4. `view::drop`

The name `view::drop` denotes a range adaptor object. Let `E` and `F` be expressions such that type `T` is `decltype((E))`.

Then, the expression `view::drop(E, F)` is expression-equivalent to:

1. `drop_view{E, F}` if `T` models `InputRange` and `F` is implicitly convertible to `iter_difference_t<iterator_t<T>>`.
2. Otherwise `view::drop(E, F)` is ill-formed.

§ 3.5. `drop_while_view`

§ 3.5.1. Motivation

The motivation for `drop_while_view` is the union of `drop_view` and `take_while_view`. Unlike the others, there are two demonstrations of `drop_while_view` below.

Current (C++17)

```
auto begin = ranges::find_if_not(employees, [](auto const holidays){ return holidays < 20; }, &employee::l  
ranges::transform(begin, ranges::end(employees), ranges::ostream_iterator<std::string>(std::cout, "\n"),  
    return e.given_name() + e.surname(); });
```

§ 3.5.2. Interface

```
template<View R, class Pred>
requires
    InputRange<R> &&
    std::is_object_v<Pred> &&
    IndirectPredicate<const Pred, iterator_t<R>>
class drop_while_view : public view_interface<drop_while_view<R, Pred>> {
public:
    drop_while_view() = default;

    constexpr drop_while_view(R base, Pred pred);

    template<ViewableRange O>
    requires constructible-from-range<R, O>
    constexpr drop_while_view(O&& o, Pred pred);

    constexpr R base() const;
    constexpr Pred pred() const;

    constexpr auto begin();
    constexpr auto end();
private:
    R base_; // exposition-only
    semiregular<Pred> pred_; // exposition-only
};

template<class R, class Pred>
drop_while_view(R&&, Pred) -> drop_while_view<all_view<R>, Pred>;
```

§ 3.5.2.1. `drop_while_view` constructors

```
constexpr drop_while_view(R base, Pred pred);
```

1. *Effects*: Initialises `base_` with `base` and initialises `pred_` with `pred`.

```
template<ViewableRange O>
requires constructible-from-range<R, O>
constexpr drop_while_view(O&& o, Pred pred)
```

2. *Effects*: Initialises `base_` with `view::all(std::forward<O>(o))`, and initialises `pred_` with `pred`.

§ 3.5.2.2. `drop_while_view` conversion

```
constexpr R base() const;
```

3. *Returns*: `base_`.

```
constexpr Pred pred() const;
```

4. *Returns*: `pred_`.

§ 3.5.2.3. `drop_while_view` begin

```
constexpr auto begin();
```

5. *Effects*: Equivalent to `return ranges::find_if_not(base_, std::ref(pred_));`.

6. *Remarks*_: In order to provide the amortized constant time complexity required by the `Range` concept, this function caches the result within the `drop_while_view` for use on subsequent calls.

§ 3.5.2.4. `drop_while_view` end

7. *Effects*: Equivalent to `return ranges::end(base_);`.

§ 3.6. `view::drop_while`

The name `view::drop_while` denotes a range adaptor object. Let `E` and `F` be expressions such that type `T` is `decltype((E))`. Then, the expression `view::drop_while(E, F)` is expression-equivalent to:

1. `drop_while_view{E, F}` if `T` models `InputRange`, and `F` is both an object and models `IndirectUnaryPredicate`.
2. Otherwise `view::drop(E, F)` is ill-formed.

§ 3.7. `basic_istream_view`

§ 3.7.1. Motivation

`istream_iterator` is an abstraction over a `basic_istream` object, so that it may be used as though it were an input iterator. It is a great way to populate a container from the get-go, or fill a container later in its lifetime. This is great, as copy is a standard algorithm that cleanly communicates that we're copying something from one range into another. There aren't any Hidden Surprises™. This is also good when writing generic code, because the generic library author does not need to care how things are inserted at the end of `v`, only that they are.

Without <code>istream_iterator</code>	With <code>istream_iterator</code>
<pre>auto v = []{ auto result = std::vector<int>{}; for (auto i = 0; std::cin >> i;) { result.push_back(i); } }(); // ... for (auto i = 0; std::cin >> i;) { result.push_back(i); }</pre>	<pre><p line-number=616>auto v = std::vector<istream_iterator<int>>{}; // ... copy(istream_iterator<int>{std::cin}, istream_iterator<int>{default_sentinel}, v);</pre>

The problem with `istream_iterator` is that we need to provide a bogus `istream_iterator<T>{} (or default_sentinel{} (or` every time we want to use it; this acts as the sentinel for `istream_iterator`. It is bogus, because the code is equivalent to saying "from the first element of the istream range until the last element of the istream range", but an `istream` range does not have a last element. Instead, we denote the end of an istream range to be when the `istream`'s failbit is set. This is otherwise known as the `_end-of-stream_` iterator value, but it doesn't denote a 'past-the-last element' in the same way that call to `vector<T>::end` does. Because it is the same for every range, the `_end-of-stream_` object may as well be dropped, so that we can write code that resembles the code below.

```
auto v = std::vector<ranges::istream_view<int>{std::cin}>{};
// ...
copy(ranges::istream_view<int>{std::cin}, back_inserter(v));
```

This code is cleaner: we are implicitly saying "until our `basic_istream` object fails, insert our input into the back of `v`". There is less to read (and type!), and the code is simpler for it.

§ 3.7.2. Interface

```
template<class T, class CharT = char, class Traits = char_traits<CharT>>
concept bool StreamExtractable = see-below;

template<class T, class charT = char, class traits = std::char_traits<charT>>
concept bool StreamInsertable = see-below;

template<Semiregular Val, class CharT, class Traits = char_traits<CharT>>
requires StreamExtractable<Val, CharT, Traits>
class basic_istream_view : public view_interface<basic_istream_view<Val, CharT, Traits>> {
public:
    basic_istream_view() = default;

    explicit constexpr basic_istream_view(std::basic_istream<CharT, Traits>& stream);

    constexpr auto begin();
    constexpr default_sentinel end() const noexcept;
private:
    struct __iterator; // exposition-only
    std::basic_istream<CharT, Traits>* stream_; // exposition-only
    Val object_; // exposition-only
};
```

§ 3.7.2.1. Concept `StreamExtractable`

```
template<class T, class CharT = char, class Traits = char_traits<CharT>>
concept bool StreamExtractable =
    requires(std::basic_istream<CharT, traits>& is, T& t) {
        { is >> t } -> Same<std::basic_istream<CharT, traits>>&;
    };
```

1. Remarks: `std::addressof(is) == std::addressof(is << t)`.

§ 3.7.2.2. Concept `StreamInsertable`

```
template<class T, class charT = char, class traits = std::char_traits<charT>>
concept bool StreamInsertable =
    requires(std::basic_ostream<charT, traits>& os, const T& t) {
        { os << t } -> Same<std::basic_ostream<charT, traits>>&;
    };
```

2. Remarks: `std::addressof(os) == std::addressof(os << t)`.

§ 3.7.2.3. `basic_istream_view` constructor

```
explicit constexpr basic_istream_view(std::basic_istream<CharT, Traits>& stream);
```

3. Effects: Initialises `stream_` to `std::addressof(stream)`.

§ 3.7.2.4. `basic_istream_view` *begin*

```
constexpr auto begin();
```

4. *Effects*: Equivalent to

```
*stream_ >> object_;  
return __iterator{*this};
```

§ 3.7.2.5. `basic_istream_view` *end*

```
constexpr default_sentinel end() const noexcept;
```

5. *Returns*: `default_sentinel{}`.

§ 3.7.2.6. `basic_istream_view::__iterator`

```
template<class Val, class CharT, class Traits>  
class basic_istream_view<Val, CharT, Traits>::__iterator {  
public:  
    using iterator_category = input_iterator_tag;  
    using difference_type = std::ptrdiff_t;  
    using value_type = Val;  
  
    __iterator() = default;  
  
    explicit constexpr __iterator(istream_view<Val>& parent) noexcept;  
  
    __iterator& operator++();  
    void operator++(int);  
  
    Val& operator*() const;  
  
    friend bool operator==(__iterator x, default_sentinel);  
    friend bool operator==(default_sentinel y, __iterator x);  
    friend bool operator!=(__iterator x, default_sentinel y);  
    friend bool operator!=(default_sentinel y, __iterator x);  
private:  
    basic_istream_view<Val, CharT, Traits>* parent_ = nullptr; // exposition-only  
};
```

§ 3.7.2.7. `basic_istream_view::__iterator` *constructor*

```
explicit constexpr __iterator(istream_view<Val>& parent) noexcept;
```

6. *Effects*: Initialises `parent_` with `std::addressof(parent_)`.

§ 3.7.2.8. `basic_istream_view::__iterator` *next*

```
__iterator& operator++();  
void operator++(int);
```

7. *Effects*: Equivalent to

```
*parent_->stream_ >> parent_->object_;
```

§ 3.7.2.9. `basic_istream_view::__iterator` value

```
Val& operator*() const;
```

8. *Effects*: Equivalent to `return parent_->value_;`

§ 3.7.2.10. `basic_istream_view::__iterator` comparison functions

```
friend bool operator==( __iterator x, default_sentinel );
```

9. *Effects*: Equivalent to `return !*x.parent_->stream_;`

```
friend bool operator==( default_sentinel y, __iterator x );
```

10. *Returns*: `x == y`.

```
friend bool operator!=( __iterator x, default_sentinel y );
```

11. *Returns*: `!(x == y)`.

```
friend bool operator!=( default_sentinel y, __iterator x );
```

12. *Returns*: `!(x == y)`.

§ 3.8. `basic_istream_view` range adaptors

The names `view::istream`, `view::wistream`, `view::u16istream`, and `view::u32istream` denote range adaptors. Let `E` be an expression and `T` be a type distinct from `decltype((E))`.

1. Then, the expression `istream_view<T>(E)` is expression-equivalent to:
2. `basic_istream_view<T, char>(E)` if `decltype((E))` models `DerivedFrom<std::istream>` and `T` models `StreamExtractable`.
3. Otherwise `istream_view<T>(E)` is ill-formed.
4. Then, the expression `wistream_view<T>(E)` is expression-equivalent to:
5. `basic_istream_view<T, wchar_t>(E)` if `decltype((E))` models `DerivedFrom<std::wistream>` and `T` models `StreamExtractable`.
6. Otherwise `wistream_view<T>(E)` is ill-formed.
7. Then, the expression `u16istream_view` is expression-equivalent to:
8. `basic_istream_view<T, char16_t>(E)` if `decltype((E))` models `DerivedFrom<basic_istream<char16_t>>` and `T` models `StreamExtractable`.
9. Otherwise `u16istream_view<T>(E)` is ill-formed.
10. Then, the expression `u32istream_view` is expression-equivalent to:
11. `basic_istream_view<T, char32_t>(E)` if `decltype((E))` models `DerivedFrom<basic_istream<char32_t>>` and `T` models `StreamExtractable`.
12. Otherwise `u32istream_view<T>(E)` is ill-formed.

§ 3.9. zip_with_view

§ 3.9.1. Motivation

A zip, also known as a [convolution](#) operation, performs a transformation on multiple input ranges. The typical zip operation transforms several input ranges into a single input range containing a tuple with the *i*th element from each range, and terminates when the smallest finite range is delimited. The following example has been adapted from [P0836](#) §2.1.

Current (C++17)	Proposed
<pre>vector<float> const x = // ... vector<float> const y = // ... float a = // ... // ... auto result = vector<float>(size(x)); transform(x, y, begin(result), [a](auto const x, auto const y) { return a * x + y; });</pre>	<pre>vector<float> const x = // ... vector<float> const y = // ... float const a = // ... // ... auto const pair_mult = [](auto const x, auto const y) { return a * x + y; }; auto const pair_plus = [](auto const x, auto const y) { return x + y; }; auto ax = view::zip(view::repeat(a), x); auto saxpy = view::zip(ax, y) view::transform(pair_plus); auto const result = vector(begin(saxpy), end(saxpy));</pre>

The benefits of this proposed approach include:

- More *declared* operations, leading to more declarative -- rather than imperative -- style programming.
- Eliminates state.
- `result` can be declared `const` without needing to rely on IILE. (Note the lack of IILE and the non-`const`ness in the former).
- Temporary storage is eliminated, which P0836 §2.1 articulates as beneficial for heterogeneous programming.

`zip_with` is a generalisation of `zip`, such that we can apply an *n*-ary function in place of the `transforms` above. The following is an example of how `zip_with` can refine `zip` when tuples are not necessary.

```
vector<float> const x = // ...
vector<float> const y = // ...
float const a = // ...
// ...
auto ax = x | view::transform([a](auto const x) noexcept { return a * x; });
auto saxpy = view::zip_with(plus<>(), ax, y);
auto const result = vector(begin(saxpy), end(saxpy));
```

Another motivating example for using `zip` involves replacing raw loops with algorithms and range adaptors, but still being able to index the operation in question.

Current (C++17)	Proposed
<pre>auto v = vector<int>(istream_iterator<int>(std::cin), istream_iterator<int>()); auto weighted_sum = 0; for (auto i = 0; static_cast<unsigned int>(i) < size(v); ++i) { weighted_sum += i * v[i]; }</pre>	<pre>auto in = view::zip(view::iota(0, size(v)), v); auto weighted_sum = accumulate(in, in + 1, 0, [](auto const [index, in]) { return a + (index * in); });</pre>

In the following text, `zip` is implemented in terms of `zip_with`.

§ 3.9.2. __common_tuple

TODO (released before San Diego).

§ 3.9.3. `__iter_zip_with`

TODO (released before San Diego).

§ 3.9.4. `zip_with_view`

TODO (released before San Diego).

§ 3.10. `view::zip_with`

The name `view::zip_with` denotes a range adaptor object. Let `Args&&...` be a parameter pack, with `args...` as its pack expansion, and let `F` be an expression. Then, the expression `view::zip_with(F, std::forward<Args>(args)...) (args)...` is expression-equivalent to:

1. `zip_with_view(F, std::forward<Args>(args)...)...`, if all parameters in the parameter pack `remove_cvref_t<Args>...` model `InputRange`, and if `F` models `RegularInvocable<iter_reference_t<iterator_t<remove_cvref_t<Args>>>...>`.
2. Otherwise, `view::zip_with(F, std::forward<Args>(args)...) (args)...` is ill-formed.

§ 3.11. `view::zip`

The name `view::zip` denotes a range adaptor object. Let `Args&&...` be a parameter pack, with `args...` as its pack expansion. Then, the expression `view::zip(std::forward<Args>(args)...) (args)...` is expression-equivalent to:

1. `view::zip_with(make_pair, std::forward<Args>(args)...)...`, if `sizeof...(Args) == 2`, and both parameters in the parameter pack `remove_cvref_t<Args>...` model `InputRange`.
2. `view::zip_with(make_tuple, std::forward<Args>(args)...)...`, if `sizeof...(Args) != 2`, and all parameters in the parameter pack `remove_cvref_t<Args>...` model `InputRange`.
3. Otherwise, `view::zip(std::forward<Args>(args)...) (args)...` is ill-formed.

§ 3.12. Reviewing `iter_value_t`, etc., and introducing `range_value_t`, etc.

P1035 also proposes to review `iter_value_t` on ranges, and to introduce a new `range_value_t`, if it is deemed appropriate.

TODO (released before San Diego).

§ References

§ Informative References

[CMCSTL2]

Casey Carter & others. [cmcstl2 C++ library](https://github.com/CaseyCarter/cmcstl2). URL: <https://github.com/CaseyCarter/cmcstl2>

[CONVOLUTION]

Wikipedia. [Convolution](https://en.wikipedia.org/wiki/Convolution_(computer_science)). URL: [https://en.wikipedia.org/wiki/Convolution_\(computer_science\)](https://en.wikipedia.org/wiki/Convolution_(computer_science))

[P0789]

Eric Niebler. [Range Adaptors and Utilities](https://wg21.link/p0789). URL: <https://wg21.link/p0789>

[P0836]

Gordon Brown; et al. [P0836 Introduce Parallelism to the Ranges TS](https://wg21.link/p0836). URL: <https://wg21.link/p0836>

[RANGE-V3]

Eric Niebler & others. [ranges-v3 C++ library](https://github.com/ericniebler/range-v3). URL: <https://github.com/ericniebler/range-v3>